

Worksheet 2 — Secure SDLC & Tooling (3 hrs)

Course: Software Security (KOSEN69) · **Week 2 Aligned to:** OWASP 2025 (A05 Injection [CWE-89, CWE-78], A04 Cryptographic Failures [CWE-327], A02 Security Misconfiguration [CWE-798, CWE-489]) · CWE-798, CWE-89, CWE-78, CWE-327, CWE-489 **Signature game:** "Bug Triage Race" (scan → triage; score = true positives – misclassified)

Ethics note: The scanners run only against the provided [vulnerable-repo/](#) on your own machine. Do not point SAST/secret scanners at third-party repos or production systems without authorization. Treat any secret you find here as fake lab data.

Part 1 — Student Information

Name	Student ID	Date	Group	AI Tool
Thura Aung	6631503094	20.8.2026	-	ChatGPT

Part 2 — Lecture Questions

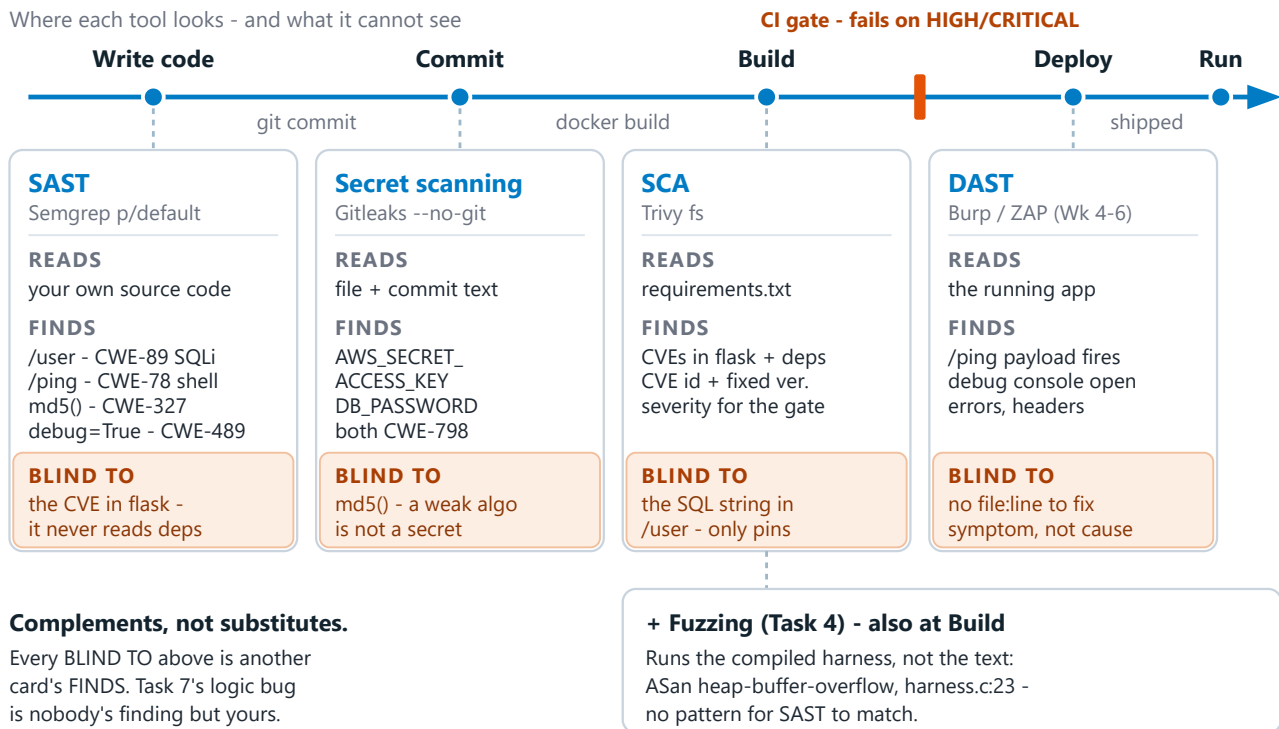
Answer in your own words (2–4 sentences each).

1. Distinguish SAST, DAST, and SCA — what does each see, and when in the SDLC does each run?
 - 1.SAST analyzes source code without running the application, so it can be used early during development. DAST tests a running application from the outside, usually during testing or staging. SCA checks third-party libraries and dependencies for known vulnerabilities and can run throughout development and CI/CD.
2. What is secret scanning, and why do hardcoded secrets keep ending up in repos?
 - 2.Secret scanning searches code and repositories for exposed passwords, API keys, tokens, and other credentials. Hardcoded secrets often appear because developers use them for testing, forget to remove them, or accidentally commit configuration files.
3. What does "shift-left / DevSecOps" mean in practice for a CI pipeline?
 - 3.Shift-left means performing security checks earlier instead of waiting until the application is finished. In CI, tools such as SAST, SCA, secret scanning, and automated tests can run on every commit or pull request so problems are found before deployment.
4. Why is coverage-guided fuzzing considered the dominant modern bug-finding technique?
 - 4.Coverage-guided fuzzing automatically generates many unusual inputs and uses code coverage to explore new execution paths. This helps it discover crashes, memory errors, and edge cases that developers may not think to test manually.
5. Define true positive vs. false positive in scanner triage, and why misclassifying both directions is costly.
 - 5.A true positive is a scanner finding that represents a real security problem, while a false positive is reported as a problem but is actually safe. Ignoring a true vulnerability creates security risk, while

treating false positives as real wastes developer time and reduces trust in security tools.

One scanner is not "the security check"

Where each tool looks - and what it cannot see



Part 3 — Hands-on Lab (180 min)

Learning goals: run a SAST tool and a secret scanner, triage findings by CWE/severity, and remediate real flaws. **Prerequisites:** Docker installed; internet to pull the Semgrep/Gitleaks images.

Environment setup

```
cd labs/week02-sdlc-tooling
cat scan.sh # see exactly what it runs
bash scan.sh # Semgrep (p/default + p/owasp-top-ten) then Gitleaks
on ./vulnerable-repo
```

Target under scan: **vulnerable-repo/app.py** (plus **requirements.txt**). It contains five planted flaws.

What to submit per task: the command/payload run + a screenshot of the finding + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* confirm tooling. *Steps:* run **bash scan.sh**; confirm both Semgrep and Gitleaks sections produce output. *Deliverable:* screenshot showing both tools ran.

(Task 0)

```

Windows PowerShell

Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call
will spawn the command using a shell process. Doing so propagates current shell settings and
variables, which makes it much easier for a malicious actor to execute commands. Use 'shell=False'
instead.
Details: https://sg.run/J92w

>> Autofix > False
26: return subprocess.check_output("ping -c 1 " + host, shell=True)

>> python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5
(( Blocking ))
Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is
therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
Details: https://sg.run/vYrY

30: return hashlib.md5(pw.encode()).hexdigest()

>> python.flask.security.audit.debug-enabled.debug-enabled
(( Blocking ))
Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will
leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug'
using system environment variables.
Details: https://sg.run/dKrd

33: app.run(debug=True) # CWE-489: debug mode in production

Scan Summary
[✓] Scan completed successfully.
• Findings: 10 (10 blocking)
• Rules run: 322
• Targets scanned: 2
• Parsed lines: ~100.0%
• No ignore information available
Ran 322 rules on 2 files: 10 findings.
PS C:\software-security\labs\week02-sdlc-tooling> Write-Output "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss K')"
```

```

Thura Aung | 6631503094 | 2026-08-27 01:34:49 +07:00
PS C:\software-security\labs\week02-sdlc-tooling>

gitleaks

Finding:      AWS_SECRET_ACCESS_KEY = "hK8pQ2mN5vX9wZ3rT6yU1sA4bC7dE0fG2hJ5kL8"
Secret:       hK8pQ2mN5vX9wZ3rT6yU1sA4bC7dE0fG2hJ5kL8
RuleID:        generic-api-key
Entropy:       5.080274
File:          /repo/app.py
Line:          11
Fingerprint:   /repo/app.py:generic-api-key:11

Finding:      DB_PASSWORD = "xQ7mK2pL9wR4tY6u"
Secret:       xQ7mK2pL9wR4tY6u
RuleID:        generic-api-key
Entropy:       4.000000
File:          /repo/app.py
Line:          12
Fingerprint:   /repo/app.py:generic-api-key:12

6:35PM INF scanned ~1139 bytes (1.14 KB) in 49.6ms
6:35PM WRN leaks found: 2

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
PS C:\software-security\labs\week02-sdlc-tooling> Write-Output "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss K')"
```

Task 1 — SAST sweep with Semgrep (25 min) · Goal: find code flaws. Steps: read the Semgrep output; locate the SQL injection in `/user` (CWE-89, string-formatted query), the OS command injection in `/ping` (CWE-78, `shell=True`), the weak `md5` password hash (CWE-327), and `debug=True` (CWE-489). Deliverable: one screenshot per finding with the file:line.

(Task 1)

- CWE-89, string-formatted query

```

/src/app.py
>> python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute
(( Blocking ))
User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and
therefore protected information could be leaked. Instead, use django's QuerySets, which are built
with query parameterization and therefore not vulnerable to sql injection. For example, you could
use 'Entry.objects.filter(date=2006)'.
Details: https://sg.run/qx7y

16: name = request.args.get("name", "")
17: con = sqlite3.connect("app.db")
18: # CWE-89: SQL injection (string formatting into query)
19: q = "SELECT * FROM users WHERE name = '%s'" % name
20: return str(con.execute(q).fetchall())
```

- ```
>>> python.flask.security.injection.subprocess-injection.subprocess-injection
((Blocking))
Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerability. An attacker could use this vulnerability to execute arbitrary commands on the host, which allows them to download malware, scan sensitive data, or run any command they wish on the server. Do not let users choose the command to run. In general, prefer to use Python API versions of system commands. If you must use subprocess, use a dictionary to allowlist a set of commands.
Details: https://sg.run/5gW3

26; return subprocess.check_output("ping -c 1 " + host, shell=True)
```

- ```
>> python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5
  (( Blocking ))
  Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is
  therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
  Details: https://sg.run/vYrY

  30: return hashlib.md5(pw.encode()).hexdigest()
```

- ```

21 python.flask.security.audit.debug-enabled.debug-enabled
 {{ Blocking }}
 Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will
 leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug'
 using system environment variables.
 Details: https://sg.run/dKrd

33! app.run(debug=True) # CWE-489: debug mode in production

```

(Task 2)

```
PS C:\software-security\labs\week02-sdlc-tooling> docker run --rm --mount type=bind,source="$(PWD.Path)\vulnerable-repo",target=/repo zricethezav/gitleaks:latest detect --source=/repo --no-git --verbose

 ○
 ○○
 ○○
 ○○
 ○○
 ○○
○
gitleaks

Finding: AWS_SECRET_ACCESS_KEY = "hK8pQ2mN5vX9wZ3rT6yU1sA4bC7dE0fG2hJ5kL8"
Secret: hK8pQ2mN5vX9wZ3rT6yU1sA4bC7dE0fG2hJ5kL8
RuleID: generic-api-key
Entropy: 5.080274
File: /repo/app.py
Line: 11
Fingerprint: /repo/app.py:generic-api-key:11

Finding: DB_PASSWORD = "xQ7mK2pL9wR4tY6u"
Secret: xQ7mK2pL9wR4tY6u
RuleID: generic-api-key
Entropy: 4.000000
File: /repo/app.py
Line: 12
Fingerprint: /repo/app.py:generic-api-key:12

6:52PM INF scanned ~1139 bytes (1.14 KB) in 46.1ms
6:52PM WRN leaks found: 2

What's next:
 Debug this container error with Gordon → docker ai "help me fix this container error"
PS C:\software-security\labs\week02-sdlc-tooling> Write-Output "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss K')"
```

## Secret Scan with Gitleaks

| Leaked credential     | File and line | Rule fired      | CWE     |
|-----------------------|---------------|-----------------|---------|
| AWS_SECRET_ACCESS_KEY | app.py:11     | generic-api-key | CWE-798 |
| DB_PASSWORD           | app.py:12     | generic-api-key | CWE-798 |

4 / 16

Mitigation: Remove the credentials from the source code and load them from environment variables or a secrets manager. Revoke and rotate any exposed credentials, and enable secret scanning in the CI pipeline to prevent future leaks.

**Task 3 — Bug Triage Race (30 min)** · *Goal:* triage accurately. *Steps:* build a table with columns *Tool* | *File:Line* | *CWE* | *Severity* | *TP/FP* | *Fix idea*; mark at least 3 true positives and 1 likely false positive and justify each. (Score = TP – misclassified.) *Deliverable:* the completed triage table.

(Task 3)

| Tool     | File:Line                 | CWE     | Severity | TP/FP                                                                                                                                                                                                                                             | Fix idea                                                                                    |
|----------|---------------------------|---------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Semgrep  | <code>app.py:19–20</code> | CWE-89  | Critical | <b>TP</b> — User input is inserted directly into an SQL query.                                                                                                                                                                                    | Use <code>con.execute("SELECT * FROM users WHERE name = ?", (name,))</code> .               |
| Semgrep  | <code>app.py:26</code>    | CWE-78  | Critical | <b>TP</b> — User-controlled <code>host</code> enters a command with <code>shell=True</code> .                                                                                                                                                     | Remove <code>shell=True</code> , pass arguments as a list, and validate the host.           |
| Semgrep  | <code>app.py:30</code>    | CWE-327 | High     | <b>TP</b> — MD5 is being used for password hashing.                                                                                                                                                                                               | Replace MD5 with Argon2id, bcrypt, or scrypt with a unique salt.                            |
| Semgrep  | <code>app.py:33</code>    | CWE-489 | High     | <b>TP</b> — Flask is explicitly started with debug mode enabled.                                                                                                                                                                                  | Disable debug mode in production and control it through environment configuration.          |
| Gitleaks | <code>app.py:11</code>    | CWE-798 | Critical | <b>TP</b> — An AWS secret key is hardcoded in the source code.                                                                                                                                                                                    | Remove and rotate the key, then load it from a secrets manager or environment variable.     |
| Gitleaks | <code>app.py:12</code>    | CWE-798 | High     | <b>TP</b> — The database password is hardcoded in the source code.                                                                                                                                                                                | Remove and rotate the password, then store it securely outside the repository.              |
| Semgrep  | <code>app.py:20</code>    | CWE-89  | Blocking | <b>Likely FP</b> — The <code>sqlalchemy-execute-raw-query</code> rule fired, but the program uses <code>sqlite3</code> , not SQLAlchemy. The underlying SQL injection is real, but this particular SQLAlchemy-specific warning is not applicable. | No separate fix is needed; parameterizing the real SQLite query fixes the underlying issue. |

(Task 4)

```

Windows PowerShell
dbd180d)
#3 0x63f76794c2a2 in fuzzer::Fuzzer::ReadAndExecuteSeedCorpora(std::vector<fuzzer::SizedFile, std::allocator<fuzzer::SizedFile>>&) (/work/fuzz+0x4f2a2) (BuildId: 09abf71c37c8bbe4c1e75952828c97701dbd180d)
#4 0x63f76794c88c in fuzzer::Fuzzer::Loop(std::vector<fuzzer::SizedFile, std::allocator<fuzzer::SizedFile>>&) (/work/fuzz+0x4f88c) (BuildId: 09abf71c37c8bbe4c1e75952828c97701dbd180d)
#5 0x63f767938e75 in fuzzer::FuzzerDriver(int*, char***, int (*)(unsigned char const*, unsigned long)) (/work/fuzz+8x3be75) (BuildId: 09abf71c37c8bbe4c1e75952828c97701dbd180d)
#6 0x63f767864672 in main (/work/fuzz+8x67672) (BuildId: 09abf71c37c8bbe4c1e75952828c97701dbd180d)
#7 0x7605f69d079 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+8x23d79) (BuildId: 48063b1a8ab5327bed56a768bf709bdba7c0844)

SUMMARY: AddressSanitizer: heap-buffer-overflow /work/harness.c:23:21 in LLVMFuzzerTestOneInput
Shadow bytes around the buggy address:
 0x501ffffffd00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x501ffffffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x501fffffff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x501fffffff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x502000000000: fa fa fd fa fa 03 fa fa fa[03]fa fa fa fa fa
 0x502000000080: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Free heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASAN internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==11==ABORTING
MS: 0 ; base unit: 0000000000000000000000000000000000
0x46,0x55,0x5a,
FUZ
artifact_prefix='./'; Test unit written to ./crash-0eb8e4ed029b774df86f2b66408203801cb982a60
Base64: RLva
root@dl14b50342a91:/work# echo "Name: Thura Aung | ID: 6631503094 | Date: $(date +%Y-%m-%d %H:%M:%S %Z)"
Name: Thura Aung | ID: 6631503094 | Date: 2026-08-26 19:26:36 UTC
root@dl14b50342a91:/work#

```

- (Task 5)

- Semgrep scan result

```

bypassing secure methods of rendering HTML by manually constructing HTML and this could create a
cross-site scripting vulnerability, which could let attackers steal sensitive user data. To be sure
this is safe, check that the HTML is rendered safely. Otherwise, use templates
('flask.render_template') which will safely render HTML instead.
Details: https://sg.run/Pb7e

284| return "<pre>%s</pre>" % out.stdout

)) python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
((Blocking))
Running flask app with host 0.0.0.0 could expose the server publicly.
Details: https://sg.run/elby

289| app.run(host="0.0.0.0", port=8080, debug=True)

)) python.flask.security.audit.debug-enabled.debug-enabled
((Blocking))
Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will
leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug'
using system environment variables.
Details: https://sg.run/dKrd

289| app.run(host="0.0.0.0", port=8080, debug=True)

Scan Summary
✔ Scan completed successfully.
• Findings: 31 (31 blocking)
• Rules run: 365
• Targets scanned: 6
• Parsed lines: ~180.0k
• No ignore information available
Ran 365 rules on 6 files: 31 findings.

A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading

Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See https://sg.run/false-positives.

```

- Gitleaks scan result

```

PS C:\software-security\labs\week02-sdlc-tooling> docker run --rm --mount type=bind,source="C:\software-security\project\starter-app",target=/repo zricethezav/gitleaks:la
test detect --source=/repo --no-git --verbose

gitleaks

9:27AM INF scanned ~11236 bytes (11.24 KB) in 54.6ms
9:27AM INF no Leaks Found
PS C:\software-security\labs\week02-sdlc-tooling> Write-Output "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss K')"
```

- SCA scan

```

2026-08-27T09:32:51Z INFO [secret] Secret scanning is enabled
2026-08-27T09:32:51Z INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-08-27T09:32:51Z INFO [secret] Please see https://trivy.dev/docs/v0.74/guide/scanner/secretrecommendation for faster secret detection
2026-08-27T09:32:51Z INFO [pip] Unable to find python site-packages directory. License detection is skipped. err="unable to find path to Python executable"
2026-08-27T09:32:51Z INFO [pip] Detecting vulnerabilities...

Report Summary
+-----+-----+-----+-----+-----+-----+-----+
| Target | Type | Vulnerabilities | Secrets |
+-----+-----+-----+-----+-----+-----+
| requirements.txt | pip | 32 | - |
+-----+-----+-----+-----+-----+-----+

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

requirements.txt (pip)
=====
Total: 32 (UNKNOWN: 0, LOW: 2, MEDIUM: 18, HIGH: 12, CRITICAL: 0)

+-----+-----+-----+-----+-----+-----+-----+
| Library | Vulnerability | Severity | Status | Installed Version | Fixed Version | Title |
+-----+-----+-----+-----+-----+-----+-----+
Flask	CVE-2023-30861	HIGH	fixed	2.0.1	2.3.2, 2.2.5	flask: Possible disclosure of permanent session cookie due to missing Wary: Cookie... https://avd.aquasec.com/nvd/cve-2023-30861	
Flask	CVE-2026-27285	LOW			3.1.3	flask: Flask: Information disclosure via improper caching of session data https://avd.aquasec.com/nvd/cve-2026-27285	
Jinja2	CVE-2024-22195	MEDIUM			3.0.1	jinja2: HTML attribute injection when passing user input as keys to mattr... https://avd.aquasec.com/nvd/cve-2024-22195	
Jinja2	CVE-2024-34064				3.1.4	jinja2: accepts keys containing non-attribute characters https://avd.aquasec.com/nvd/cve-2024-34064	
Jinja2	CVE-2024-56201				3.1.5	jinja2: Jinja has a sandbox breakout through malicious filenames https://avd.aquasec.com/nvd/cve-2024-56201	
Jinja2	CVE-2024-56326					jinja2: Jinja has a sandbox breakout through indirect reference to format method... https://avd.aquasec.com/nvd/cve-2024-56326	
Jinja2	CVE-2025-27516				3.1.6	jinja2: Jinja sandbox breakout through attr filter selecting format method https://avd.aquasec.com/nvd/cve-2025-27516	
PyJWT	CVE-2022-29217	HIGH			1.7.1	2.4.0	python-jwt: Key confusion through non-blocklisted public key formats https://avd.aquasec.com/nvd/cve-2022-29217
PyJWT	CVE-2026-32597					pyjwt: PyJWT accepts unknown 'crit' header extensions (RFC 7615 §4.1.11 MUST violation)... https://avd.aquasec.com/nvd/cve-2026-32597	
PyJWT	CVE-2026-48526					python-pyjwt: PyJWT: Authentication bypass due to forged JSON web tokens https://avd.aquasec.com/nvd/cve-2026-48526	

```

- Semgrep findings (Duplicate Semgrep alerts were consolidated into distinct vulnerabilities.)

| Tool    | File:Line     | Finding                   | CWE     |
|---------|---------------|---------------------------|---------|
| Semgrep | Dockerfile:12 | Container may run as root | CWE-250 |



| Tool    | File:Line                            | Finding                                               | CWE                |
|---------|--------------------------------------|-------------------------------------------------------|--------------------|
| Semgrep | <a href="#">app.py:68–69,117,129</a> | MD5 password hashing                                  | CWE-916 / CWE-327  |
| Semgrep | <a href="#">app.py:83</a>            | JWT accepts the <a href="#">none</a> algorithm        | CWE-347            |
| Semgrep | <a href="#">app.py:106</a>           | Potential unsafe template rendering                   | CWE-79             |
| Semgrep | <a href="#">app.py:128–130</a>       | SQL injection in login query                          | CWE-89             |
| Semgrep | <a href="#">app.py:134</a>           | Hardcoded JWT signing secret                          | CWE-798            |
| Semgrep | <a href="#">app.py:136</a>           | Cookie missing security attributes                    | CWE-614 / CWE-1004 |
| Semgrep | <a href="#">app.py:176–179</a>       | SQL injection in note search                          | CWE-89             |
| Semgrep | <a href="#">app.py:181–182</a>       | User data inserted into raw HTML                      | CWE-79             |
| Semgrep | <a href="#">app.py:202–203</a>       | OS command injection using <a href="#">shell=True</a> | CWE-78             |
| Semgrep | <a href="#">app.py:204</a>           | Unescaped formatted response                          | CWE-79             |
| Semgrep | <a href="#">app.py:209</a>           | Application exposed on all interfaces                 | CWE-668            |
| Semgrep | <a href="#">app.py:209</a>           | Flask debug mode enabled                              | CWE-489            |

- Trivy SCA findings (All findings came from outdated packages in requirements.txt.)

| Package  | CVE                            | Severity | CWE                |
|----------|--------------------------------|----------|--------------------|
| Flask    | <a href="#">CVE-2023-30861</a> | High     | CWE-539            |
| Flask    | <a href="#">CVE-2026-27205</a> | Low      | CWE-524            |
| Jinja2   | <a href="#">CVE-2024-22195</a> | Medium   | CWE-79             |
| Jinja2   | <a href="#">CVE-2024-34064</a> | Medium   | CWE-79             |
| Jinja2   | <a href="#">CVE-2024-56201</a> | Medium   | CWE-150            |
| Jinja2   | <a href="#">CVE-2024-56326</a> | Medium   | CWE-693 / CWE-1336 |
| Jinja2   | <a href="#">CVE-2025-27516</a> | Medium   | CWE-1336           |
| PyJWT    | <a href="#">CVE-2022-29217</a> | High     | CWE-327            |
| PyJWT    | <a href="#">CVE-2026-32597</a> | High     | CWE-345 / CWE-347  |
| PyJWT    | <a href="#">CVE-2026-48526</a> | High     | CWE-287 / CWE-347  |
| Werkzeug | <a href="#">CVE-2023-25577</a> | High     | CWE-770            |
| Werkzeug | <a href="#">CVE-2024-34069</a> | High     | CWE-352            |
| Werkzeug | <a href="#">CVE-2023-46136</a> | Medium   | CWE-400 / CWE-407  |



| Package  | CVE                            | Severity | CWE               |
|----------|--------------------------------|----------|-------------------|
| Werkzeug | <a href="#">CVE-2024-49766</a> | Medium   | CWE-22            |
| Werkzeug | <a href="#">CVE-2024-49767</a> | Medium   | CWE-400 / CWE-770 |
| Werkzeug | <a href="#">CVE-2025-66221</a> | Medium   | CWE-67            |
| Werkzeug | <a href="#">CVE-2026-21860</a> | Medium   | CWE-67            |
| Werkzeug | <a href="#">CVE-2026-27199</a> | Medium   | CWE-67            |
| Werkzeug | <a href="#">CVE-2023-23934</a> | Low      | CWE-20            |
| requests | <a href="#">CVE-2023-32681</a> | Medium   | CWE-200           |
| requests | <a href="#">CVE-2024-35195</a> | Medium   | CWE-670           |
| requests | <a href="#">CVE-2024-47081</a> | Medium   | CWE-522           |
| requests | <a href="#">CVE-2026-25645</a> | Medium   | CWE-377           |
| urllib3  | <a href="#">CVE-2021-33503</a> | High     | CWE-400           |
| urllib3  | <a href="#">CVE-2023-43804</a> | High     | CWE-200           |
| urllib3  | <a href="#">CVE-2025-66418</a> | High     | CWE-770           |
| urllib3  | <a href="#">CVE-2025-66471</a> | High     | CWE-409           |
| urllib3  | <a href="#">CVE-2026-21441</a> | High     | CWE-409           |
| urllib3  | <a href="#">CVE-2026-44431</a> | High     | CWE-200           |
| urllib3  | <a href="#">CVE-2023-45803</a> | Medium   | CWE-200           |
| urllib3  | <a href="#">CVE-2024-37891</a> | Medium   | CWE-669           |
| urllib3  | <a href="#">CVE-2025-50181</a> | Medium   | CWE-601           |

**Task 6 — Build a security CI gate (25 min)** · *Goal:* automate the scan (previews Week 15). *Steps:* adapt [../week15-devsecops-pipeline/security-ci.yml](#) into a workflow that runs Semgrep + Trivy + Gitleaks and **fails on HIGH/CRITICAL**; run it locally ([act](#)) or commit to your fork and read the Actions log. *Deliverable:* the workflow file + a screenshot of a failing run.

(Task 6)

- the workflow file

```
name: NoteVault Security CI

on:
 push:
 branches: [main]
 paths:
 - "project/starter-app/**"
 - ".github/workflows/notevault-security-ci.yml"
```

```
pull_request:
 branches: [main]
workflow_dispatch:

permissions:
 contents: read

jobs:
 security-gate:
 name: Semgrep + Trivy + Gitleaks
 runs-on: ubuntu-latest

 steps:
 - name: Check out repository
 uses: actions/checkout@v4

 - name: Run Semgrep SAST
 id: semgrep
 continue-on-error: true
 run: |
 docker run --rm \
 -v "$PWD/project/starter-app:/src" \
 semgrep/semgrep:latest \
 semgrep scan \
 --config p/default \
 --config p/owasp-top-ten \
 --error /src

 - name: Run Trivy SCA gate
 id: trivy
 continue-on-error: true
 uses: aquasecurity/trivy-action@ed142fd0673e97e23eac54620cfb913e5ce36c25
 with:
 scan-type: fs
 scan-ref: project/starter-app
 scanners: vuln
 severity: HIGH,CRITICAL
 ignore-unfixed: false
 exit-code: "1"

 - name: Run Gitleaks secret scan
 id: gitleaks
 continue-on-error: true
 run: |
 docker run --rm \
 -v "$PWD/project/starter-app:/repo" \
 zricethezav/gitleaks:latest \
 detect \
 --source=/repo \
 --no-git \
 --verbose \
 --exit-code 1

 - name: Enforce security gate
```

```

if: always()
env:
 SEMGREP_OUTCOME: ${ steps.semgrep.outcome }
 TRIVY_OUTCOME: ${ steps.trivy.outcome }
 GITLEAKS_OUTCOME: ${ steps.gitleaks.outcome }
run: |
 echo "Semgrep: $SEMGREP_OUTCOME"
 echo "Trivy: $TRIVY_OUTCOME"
 echo "Gitleaks: $GITLEAKS_OUTCOME"

 if [["$SEMGREP_OUTCOME" == "failure" ||
 "$TRIVY_OUTCOME" == "failure" ||
 "$GITLEAKS_OUTCOME" == "failure"]]; then
 echo "::error::Security gate failed because vulnerabilities or secrets
were detected."
 exit 1
 fi

 echo "Security gate passed."

```

[NoteVault security CI gate failing on security findings]

The screenshot shows a GitHub Actions workflow run for 'NoteVault Security CI'. The workflow is titled 'Fix Trivy action version in security gate #2'. The status is 'Failure'. The job 'Semgrep + Trivy + Gitleaks' failed with an exit code of 1. The annotations section shows 4 errors and 1 warning, with the first error being 'Semgrep + Trivy + Gitleaks: Process completed with exit code 1.'

**Task 7 — SAST blind spots (20 min)** · *Goal:* see what scanners miss. *Steps:* find one real bug in [vulnerable-repo/app.py](#) (or NoteVault) that Semgrep did **not** flag, and explain why a pattern-based tool missed it. *Deliverable:* the bug + a 2-sentence explanation.

(Task 7)

- Bug: IDOR / broken object-level authorization in project/starter-app/app.py:161–168 (CWE-639). The /api/notes/ endpoint checks only whether the requester is logged in, but it does not verify that the requested note belongs to that user.
- Explanation

Semgrep missed this because the SQL query is parameterized and does not match a dangerous injection pattern. Detecting the bug requires understanding the application's authorization rule—that users should access only their own notes—which a pattern-based scanner may not know.

**Task 8 — Defend / fix it (10 min)** · *Goal:* remediate the planted flaws in `vulnerable-repo/app.py`. *Steps:* rewrite `/user` to use a parameterized query (`?` placeholder); remove `shell=True` and pass an argument list in `/ping`; move both secrets to environment variables; replace `md5` with `bcrypt/argon2`; set `debug=False`. *Deliverable:* a before/after diff for each fix mapped to its CWE.

(Task 8)

- Fixed code

```
"""
Secured Week 2 sample application.
"""

import ipaddress
import os
import sqlite3
import subprocess

from argon2 import PasswordHasher
from flask import Flask, request

app = Flask(__name__)
password_hasher = PasswordHasher()

CWE-798 fixed: secrets are loaded from environment variables.
AWS_SECRET_ACCESS_KEY = os.environ["AWS_SECRET_ACCESS_KEY"]
DB_PASSWORD = os.environ["DB_PASSWORD"]

@app.route("/user")
def user():
 name = request.args.get("name", "")
 con = sqlite3.connect("app.db")

 # CWE-89 fixed: parameterized SQL query.
 rows = con.execute(
 "SELECT * FROM users WHERE name = ?",
 (name,),
).fetchall()

 con.close()
 return str(rows)

@app.route("/ping")
def ping():
 supplied_host = request.args.get("host", "127.0.0.1")
```

```

try:
 host = str(ipaddress.ip_address(supplied_host))
except ValueError:
 return "Invalid IP address", 400

CWE-78 fixed: validated input, argument list, and no shell=True.
return subprocess.check_output(
 ["ping", "-c", "1", host],
 text=True,
)

def store_password(password):
 # CWE-916/CWE-327 fixed: PasswordHasher uses Argon2id.
 return password_hasher.hash(password)

if __name__ == "__main__":
 app.run(debug=False) # CWE-489 fixed

```

- Before/after fixes

| CWE                  | Before                                                     | After                                                                           |
|----------------------|------------------------------------------------------------|---------------------------------------------------------------------------------|
| CWE-89               | <code>q = "SELECT ... '%s'" % name</code>                  | <code>con.execute("SELECT ... ?", (name,))</code>                               |
| CWE-78               | <code>check_output("ping -c 1 " + host, shell=True)</code> | Validate the IP and use<br><code>check_output(["ping", "-c", "1", host])</code> |
| CWE-798              | Secrets stored directly in <code>app.py</code>             | Load both secrets with <code>os.environ[...]</code>                             |
| CWE-916 /<br>CWE-327 | <code>hashlib.md5(pw.encode()).hexdigest()</code>          | <code>PasswordHasher().hash(password)</code> using Argon2id                     |
| CWE-489              | <code>app.run(debug=True)</code>                           | <code>app.run(debug=False)</code>                                               |

## Part 4 — Reflection

1. Map two of your findings to their CWE and to the matching OWASP 2025 category.

The SQL injection in `/user` maps to CWE-89 and OWASP 2025 A05: Injection. The hardcoded AWS key and database password map to CWE-798 and OWASP 2025 A07: Authentication Failures.

2. Name a real-world breach caused by a hardcoded/leaked secret or an injection flaw, and what control would have caught it pre-release.

In Uber's 2016 breach, attackers found an AWS access key stored in a private GitHub repository and used it to access data in Amazon S3. Automated secret scanning on every commit and pull request could have detected and blocked the exposed key before release.

3. Which single tool (SAST vs. secret scanning) gave the highest-value findings on this repo, and why?

SAST provided the highest-value findings because Semgrep identified several directly exploitable weaknesses, including SQL injection, command injection, insecure MD5 password hashing, and debug mode. Secret scanning was still valuable, but SAST revealed more distinct vulnerabilities and explained the unsafe code lines that required correction.

## Grading rubric (100)

| Criterion                                                          | Points |
|--------------------------------------------------------------------|--------|
| Lecture questions (Part 2)                                         | 20     |
| Exploitation + evidence (scan output + triage table + screenshots) | 40     |
| Defense (remediated <code>app.py</code> with before/after diffs)   | 25     |
| Reflection (CWE/OWASP mapping + breach + tool value)               | 15     |

## Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence.** When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** \_\_\_\_\_ *Flags are unique per student — submitting another student's flag is a violation. How to submit: [learn.zcr.ai/submit](https://learn.zcr.ai/submit) (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):
  1. What did you do, and **why did the vulnerability work**?
  2. **Why does your fix actually stop it** — and what could still break it?



## Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

1. AI assistant's full answer

- To fix the SQL injection, escape single quotation marks before adding the username to the query:

```
@app.route("/user")
def user():
 name = request.args.get("name", "")
 safe_name = name.replace("'", "'")
 con = sqlite3.connect("app.db")
 q = f"SELECT * FROM users WHERE name = '{safe_name}'"
 return str(con.execute(q).fetchall())
```

- Escaping quotation marks prevents attackers from breaking out of the SQL string.

---

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

---

2. What is wrong or risky

- The risky lines are:

```
safe_name = name.replace("'", "'")
q = f"SELECT * FROM users WHERE name = '{safe_name}'"
```

---

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

---

3. Correct, verified version

```
@app.route("/user")
def user():
 name = request.args.get("name", "")
 con = sqlite3.connect("app.db")
 rows = con.execute(
 "SELECT * FROM users WHERE name = ?",
 (name,),
).fetchall()
 con.close()
 return str(rows)
```

The AI's fix was insufficient because it continued mixing user input with SQL and relied on hand-written escaping. The parameterized version sends the query and username separately, so malicious text is treated only as data. After verification, the SQL-injection Semgrep warning disappeared, and the payload ' OR 1=1 -- no longer returned unintended users.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.



## Comprehension & Prompt (required)

**A. Explain in Plain English (EiPE).** In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

- The /user endpoint takes a name from the URL and searches the database for a matching user. The vulnerable version inserted that input directly into the SQL command, so an attacker could submit SQL syntax such as ' OR 1=1 -- and make the database return records they were not supposed to see.

**B. Prompt Problem.** Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

- Final prompt given to the AI:

Fix only the SQL injection vulnerability in the Flask /user endpoint. Use Python's sqlite3 parameterized-query syntax with a literal ? placeholder and pass name as the one-element tuple (name,). Do not use string formatting, concatenation, f-strings, manual escaping, or an ORM. Preserve the endpoint's existing behavior, fetch the results, close the database connection, and return the corrected Python code.

AI-generated secure fix:

```
@app.route("/user")
def user():
 name = request.args.get("name", "")
 con = sqlite3.connect("app.db")

 rows = con.execute(
 "SELECT * FROM users WHERE name = ?",
 (name,),
).fetchall()

 con.close()
 return str(rows)
```

Verified result:

I tested the fixed endpoint with the payload ' OR 1=1 --. It returned [] instead of returning every database record, while the normal input alice returned only Alice's record; therefore, the SQL injection exploit failed. A new Semgrep scan also no longer reported the original CWE-89 SQL-injection finding.